

EVO-RAG: Evolving Retrieval-Augmented Agents for Efficient Multi-Hop Query Optimization

Yuelyu Ji¹, Rui Meng², Zhuochun Li¹, Daqing He¹

¹School of Computing and Information, University of Pittsburgh

²Google Cloud AI Research, Sunnyvale, CA, USA

Abstract

Retrieval-augmented generation (RAG) grounds large language models (LLMs) in external evidence, yet *multi-hop* pipelines still suffer from redundant sub-queries, shallow exploration, and premature or delayed stopping. We present EVO-RAG, a phase-aware framework that couples a lightweight two-stage curriculum (**Discovery**→**Refinement**) with **seven step-level rewards** and an **in-episode time scheduler**. The scheduler decays exploration incentives as evidence accumulates while increasing efficiency and correctness pressure as uncertainty shrinks. Beyond scalar rewards, we train a **multi-head preference model** and benchmark **DPO**, **PPO**, and **GRPO** under *identical* rollouts and curricula for a controlled comparison. Evaluated on HotpotQA, 2WikiMultiHopQA, MuSiQue, and Bamboogle with 8B-class backbones, EVO-RAG improves EM/F1 while reducing redundant hops. Ablations show that (i) suppressing query overlap, (ii) rewarding *controlled backtracking* and *justified refusal*, and (iii) time-dynamic weighting are key to the accuracy–efficiency trade-off.

1 Introduction

Large language models (LLMs) deliver strong results in QA, dialogue, and text generation (Brown et al., 2020; Ouyang et al., 2022; Raffel et al., 2020), yet they still hallucinate when relying on static pretraining. Retrieval-Augmented Generation (RAG) grounds responses in external documents (Lewis et al., 2020), but *multi-hop* QA remains difficult: an agent must issue a *sequence* of sub-queries, integrate partial evidence, and decide when to backtrack, answer, or refuse.

Modern RAG pipelines span query rewriting, retrieval, filtering/reranking, and answer generation (Chen et al., 2024b; Gao et al., 2024). End-to-end objectives that couple retriever and generator reduce handoff errors (Chen et al., 2024b; Gao et al., 2024; Xiong et al., 2025), but most supervision

is *static* and phase-agnostic. As a result, systems often over-search early or fail to consolidate late. RL-based approaches attempt to align modules to task rewards, yet many depend on episode-level signals and fixed weight schedules, offering weak credit assignment for intermediate actions and poor guidance on the transition from *exploration* to *refinement* (Huang et al., 2025; Song et al., 2025; Liu et al., 2025; Sun et al., 2025).

We introduce EVO-RAG, a phase-aware framework for multi-hop retrieval (Fig. 1). The agent operates in two stages—*Discovery* then *Refinement*—and receives seven interpretable *step-level* signals: retrieval hit/miss, retrieval-action penalty, sub-query overlap, backtrack, refusal validity, step cost, and answer correctness. A time-based scheduler adjusts signal weights *within each episode*, decaying exploration incentives as evidence accumulates while increasing efficiency and correctness pressure as uncertainty shrinks. Beyond scalar rewards, we build a multi-head preference model that scores trajectory prefixes along these aspects, enabling both preference alignment (DPO) and scalarized policy gradients (PPO/GRPO) under identical rollouts.

Our design couples two time scales. Across training, a lightweight two-stage curriculum sets *endpoints* for each reward weight (discovery, then refinement) without freezing behavior; within episodes, linear interpolation by progress ratio $p(t)$ produces smooth, phase-aware guidance. This separation keeps implementation simple—no additional modules at inference time—while providing dense, interpretable feedback for control-flow decisions such as BACKTRACK and REFUSE.

We evaluate on HotpotQA, 2WikiMultiHopQA, MuSiQue, and Bamboogle (Yang et al., 2018; Ho et al., 2020; Trivedi et al., 2022b; Press et al., 2022) using 8B-class backbones. To probe generalization, we train on a small HotpotQA subset and test across datasets. EVO-RAG improves EM/F1 while curb-

ing redundant hops; ablations show that (i) suppressing query overlap, (ii) rewarding controlled backtracking and justified refusal, and (iii) time-based weighting are key to the accuracy–efficiency trade-off. We also report a controlled comparison of DPO, PPO, and GRPO on the *same* rollouts and curricula, surfacing objective-dependent differences without confounds. Our experimental emphasis is a controlled analysis of phase-aware step-level supervision and policy objectives under matched rollouts, rather than an exhaustive comparison against every recent search-augmented RAG variant.

2 Related Work

RAG paradigms. RAG combines retrieval with generation to improve factuality and reduce hallucinations (Lewis et al., 2020). Comparative studies map retriever–generator trade-offs and explore end-to-end/process supervision that tightens interaction between the two (Chen et al., 2024b; Gao et al., 2024; Xiong et al., 2025). These objectives are typically *static* and phase-agnostic; our approach provides step-level, time-scheduled guidance within an episode to reflect evolving information needs in multi-hop reasoning.

Query rewriting and multi-hop retrieval. Multi-hop QA requires issuing well-formed sub-queries conditioned on partial evidence; errors propagate if later hops inherit poor queries. Methods mitigate this with missing-entity completion, interleaved reasoning–retrieval (e.g., IRCOT), speculative querying, and coherence-aware reranking (Trivedi et al., 2022a; Wang et al., 2024; Zhang et al., 2024; Wei et al., 2024; Zhu et al., 2025). These approaches seldom supervise *when* to diversify versus consolidate. Our overlap and step-cost signals explicitly regulate duplication and chain length, while backtrack/refusal signals shape control flow.

RL and preference optimization for RAG. RL has been used to align retrieval, reranking, and generation (e.g., multi-agent or curriculum-based training) and to leverage preference objectives such as DPO/GRPO (Chen et al., 2025; Huang et al., 2025; Ramesh et al., 2024; Kaiser and Weikum, 2025; Liu et al., 2025). Most rely on episode-level rewards with fixed schedules, which weakens credit assignment for intermediate actions. EVO-RAG instead supplies seven *step-level* signals with an

in-episode scheduler and reports DPO/PPO/GRPO under identical rollouts for a controlled comparison.

3 Method

3.1 Preliminaries: Agentic RAG on RAG-Gym

We build on the high-level MDP abstraction of agentic RAG popularized by prior toolkits (“RAG-Gym” (Xiong et al., 2025)). Each episode corresponds to one question x and unfolds as a sequence $(s_t, a_t, o_t)_{t=1}^T$: **State** s_t : the question, the history of sub-queries and retrieved snippets, and the current scratchpad. **Observation** o_t : the top- k passages returned by the IR system when $a_t = \text{SEARCH}$, or $\emptyset$ for other actions. **Action space**:

$$\mathcal{A} = \{\text{SEARCH}, \text{BACKTRACK}, \text{ANSWER}, \text{REFUSE}\}.$$

Termination: when the agent emits ANSWER or REFUSE, or $t = T_{\max}$. This paper keeps the outcome reward on the final ANSWER (EM/F1) but *decomposes* process feedback at intermediate steps into seven interpretable signals (Sec. 3.4). A two-stage curriculum (Discovery→Refinement) and an in-episode time scheduler (Sec. 3.7) shape the relative weights of these signals.

3.2 Why these actions and signals?

Multi-hop RAG agents fail in a few recurring ways: (i) issuing paraphrased sub-queries that retrieve near-duplicate evidence, (ii) over-searching late and polluting the context window, (iii) getting stuck after a retrieval miss and failing to recover, and (iv) hallucinating answers when evidence is insufficient. EVO-RAG exposes four control actions to directly address these failure modes: SEARCH expands evidence, BACKTRACK recovers from dead-ends, ANSWER commits when evidence is sufficient, and REFUSE abstains when the question is not answerable given retrieved evidence. We design seven step-level signals to provide observable, low-overhead feedback aligned with these behaviors: retrieval hit/miss encourages early grounding; overlap and step cost discourage redundancy and long chains; action penalty discourages late, redundant searching; backtrack penalty regularizes reversibility; refusal validity prevents unsafe abstention; and answer correctness provides the final task objective.

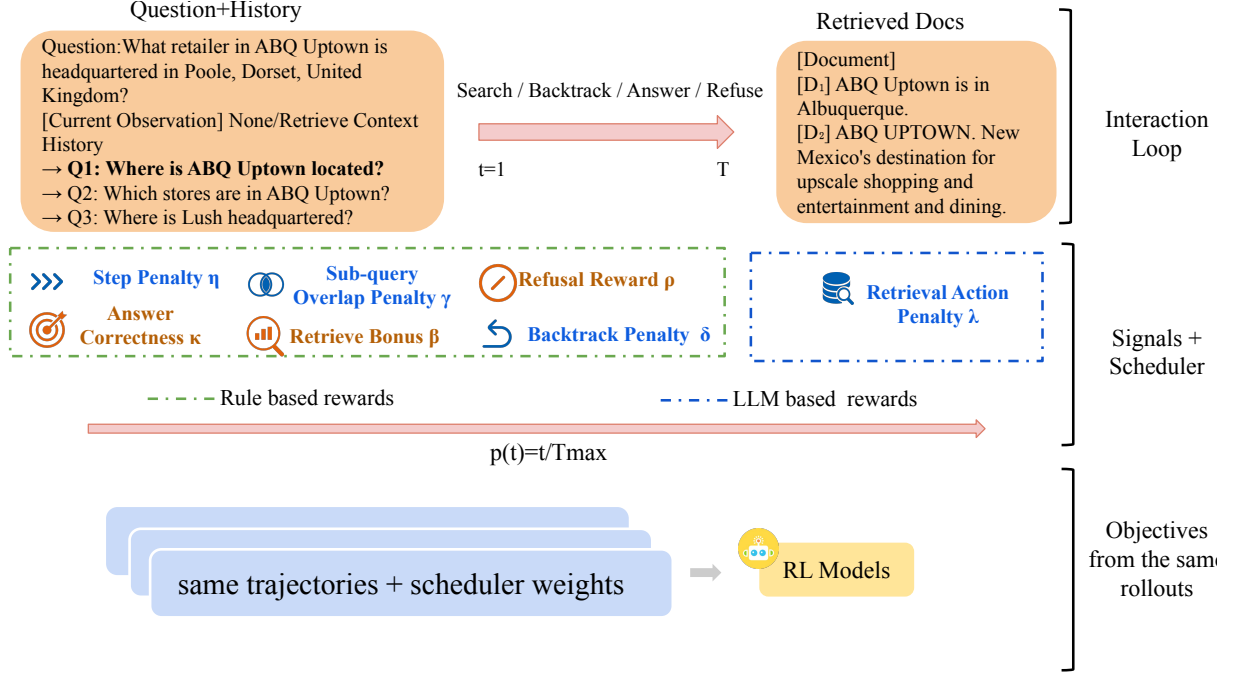


Figure 1: **EVO-RAG overview.** Left: at hop t the agent rewrites a sub-query and retrieves evidence; four actions are available (continue, backtrack, answer, refuse). Right: seven step-level signals—*Retrieval Bonus*, *Retrieval Action Penalty*, *Sub-query Overlap Penalty*, *Backtrack Penalty*, *Refusal Reward*, *Step Penalty*, *Answer Correctness*—with a time-based scheduler that shifts emphasis from exploration to refinement. Lower: we train the same rollouts with three policy objectives (DPO/PPO/GRPO) for transparent per-dataset/per-backbone comparison. Only refusal validity uses an external LLM judge; all other signals are computed from the environment / trajectory statistics.

3.3 Reward Sources and Action Triggers

We use two feedback sources: (i) *rule-based* signals computable from the environment (hit gold doc, query redundancy, step cost), and (ii) *LLM-based* judgments for semantics-heavy cases (whether the current evidence suffices, thus REFUSE is justified). Table 1 summarizes *when* each signal fires and *who* provides it; formal definitions follow in Sec. 3.4.

Refusal validity judge. When the policy selects REFUSE, we validate whether the question is answerable *using only the currently retrieved evidence* with an external LLM judge. We use **GPT-4.1** (temperature 0) and a short rubric prompt (Appendix E). The judge is called *only* on the REFUSE action (one call per refusal) and outputs a single label in {answerable, unanswerable}. This design keeps judge usage minimal and prevents it from affecting non-refusal trajectories.

3.4 Step-level Reward

Overlap statistic.

$$\Delta_t = \max_{j < t} \cos(q_t, q_j).$$

Table 1: Process signals, trigger, and source. All symbols are defined in Sec. 3.4.

Signal	Action / Timing	Source	Intuition
r_{ret}	SEARCH	Rule	Reward early hits on D^* .
r_{dup}	SEARCH	Rule	Penalize redundant sub-queries.
r_{bt}	BACKTRACK	Rule	Discourage blind backtracking.
r_{ref}	REFUSE	LLM judge	Refuse when evidence is insufficient.
r_{step}	every step	Rule	Keep chains short.
r_{act}	late SEARCH	Rule	Curb late redundant searches.
r_{ans}	terminal ANSWER	Rule	EM/F1 w.r.t. A^* .

Retrieval Bonus (r_{ret}). Let D_t be retrieved documents at step t and D^* the gold-supporting set.

$$r_{\text{ret}}(s_t, a_t) = \mathbb{I}[a_t = \text{SEARCH}] \cdot (2\mathbb{I}[D_t \cap D^* \neq \emptyset] - 1).$$

Training-time oracle signals vs. inference-time behavior. The gold supporting set D^* is provided by datasets with annotated supporting documents (e.g., HotpotQA/2Wiki/MuSiQue) and is used *only during training* to compute r_{ret} (and for analysis).

At inference time, D^* is unavailable; the deployed policy conditions solely on the question and retrieved passages, and no oracle-based reward is used. Thus, our method does not require supervision signals at test time and introduces no information leakage into inference.

Sub-query Overlap Penalty (r_{dup}).

$$r_{\text{dup}}(s_t, a_t) = -\mathbb{I}[a_t = \text{SEARCH}] \cdot \Delta_t.$$

Backtrack Penalty (r_{bt}).

$$r_{\text{bt}}(s_t, a_t) = -\mathbb{I}[a_t = \text{BACKTRACK}].$$

Refusal Reward (r_{ref}). Let $J_{\text{ref}}(s_t) \in \{0, 1\}$ be the external judge output given current evidence, where 1 marks an unanswerable question and 0 an answerable one.

$$r_{\text{ref}}(s_t, a_t) = \mathbb{I}[a_t = \text{REFUSE}] \cdot (2J_{\text{ref}}(s_t) - 1).$$

Step Cost (r_{step}).

$$r_{\text{step}}(s_t, a_t) = -1.$$

Answer Correctness (r_{ans}).

$$r_{\text{ans}}(s_T, a_T) = \mathbb{I}[a_T = \text{ANSWER}] \cdot \frac{1}{2} (\text{EM}(A_T, A^*) + \text{F1}(A_T, A^*)).$$

Retrieval Action Penalty (r_{act}). We penalize redundant late searches only after progress $p(t) \geq 0.3$:

$$r_{\text{act}}(s_t, a_t) = -\mathbb{I}[a_t = \text{SEARCH}] \mathbb{I}[p(t) \geq 0.3] \mathbb{I}[\Delta_t \geq \tau].$$

Total scalarized reward. Let $K = 7$ be the number of step signals above. The per-step scalar reward is

$$R_t = \sum_{k=1}^K w_k(t) r_t^{(k)},$$

where the weights $w_k(t)$ are scheduled as described in Sec. 3.7.

3.5 Preference Modeling & Policy Objectives

Multi-head preference model. Given rollouts with step-level labels $\{r_t^{(k)}\}_{k=1}^7$ and time weights $\{w_k(t)\}$, we construct preference pairs (x^+, x^-) at the *trajectory-prefix* level using the weighted return $\sum_t \sum_k w_k(t) r_t^{(k)}$. A shared encoder with

seven linear heads $\{f_\phi^{(k)}\}_{k=1}^7$ scores each aspect; the head-wise pairwise loss is

$$\mathcal{L}_{\text{RM}} = -\frac{1}{7} \sum_{k=1}^7 \log \sigma(f_\phi^{(k)}(x^+) - f_\phi^{(k)}(x^-)),$$

where $\sigma(\cdot)$ is the logistic function. This factorization preserves interpretability and allows either preference- or reward-based policy learning.

Path A: preference alignment (DPO). We feed (x^+, x^-) directly to the policy and optimize

$$\mathcal{L}_{\text{DPO}} = -\log \sigma(\beta_{\text{dpo}} [\log \pi_\theta(x^+) - \log \pi_\theta(x^-)]).$$

No scalarization is required.

Path B: scalarized policy gradients (PPO / GRPO). When desired, aspect scores (or environment labels) are linearly combined into a scalar per-step reward:

$$\tilde{r}_t = \sum_k w_k(t) r_t^{(k)}.$$

We compute advantages with GAE and apply the PPO objective

$$\mathcal{L}_{\text{PPO}} = -\mathbb{E}_t \left[\min \left(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t \right) \right],$$

and its group-normalized variant (GRPO) by replacing A_t with

$$\hat{A}_t^{(i)} = \frac{\tilde{r}_t^{(i)} - \mu_t}{\sigma_t + \epsilon} \quad \text{over candidates } i.$$

Objective summary. We report all three objectives under identical rollouts and curricula. DPO consumes preferences; PPO/GRPO use the same per-step weights for scalarization. We refrain from universal claims; effects depend on dataset/backbone and are analyzed in Sec. 4.5.

Retrieval-focused weights (β, λ) monotonically decrease, whereas efficiency-focused weights (γ, η, κ) increase; the refusal weight ρ stays constant. This “gearbox” provides step-level guidance that is missing from a static two-stage switch.

3.6 Two-Stage Curriculum (across training)

We use two time scales for guidance: a *training-time* two-stage curriculum (Discovery $\rightarrow$ Refinement) and an *in-episode* time-based scheduler (Sec. 3.7). A “stage” does *not* fix weights; it only specifies the *interpolation endpoints*—Start $\rightarrow$ Mid in Discovery and Mid $\rightarrow$ End in Refinement—while the actual step-wise weights still evolve within each episode via $p(t)$.

Table 2: Reward weights for EVO-RAG training. Start→Mid are the interpolation endpoints used in Stage 1 (Discovery); Mid→End are those used in Stage 2 (Refinement).

Reward Component	Stage 1: Discovery			Stage 2: Refinement		
	Start	Mid	Trend	Mid	End	Trend
Retrieval Bonus (β)	2.0	1.0	↘	1.0	0.5	↘
Retrieval Act. Pen. (λ)	1.5	0.8	↘	0.8	0.4	↘
Overlap Pen. (γ)	0.1	0.5	↗	0.5	1.2	↗
Backtrack Pen. (δ)	0.3	0.5	↗	0.5	1.0	↗
Refusal Reward (ρ)	0.5	0.5	—	0.5	0.5	—
Step Penalty (η)	0.02	0.05	↗	0.05	0.10	↗
Answer Corr. (κ)	0.05	0.10	↗	0.10	1.00	↗

What each stage emphasizes. **Discovery** exposes the policy to a high-entropy evidence space. We therefore set larger Start–Mid endpoints for retrieval-oriented terms (w_β, w_λ) and smaller ones for efficiency/precision ($w_\gamma, w_\eta, w_\kappa$) to encourage breadth and early hits. **Refinement** shifts these endpoints in the opposite direction (larger $w_\gamma, w_\eta, w_\kappa$, smaller w_β, w_λ), promoting consolidation, controlled stopping, and precise answering. The refusal weight w_ρ stays constant across stages so that safe refusal is always available.

When to switch stages. We switch from Discovery to Refinement once exploration no longer increases the *composite return*. Concretely, let

$$R(\tau) = \sum_{t=1}^T \sum_k w_k(t) r_t^{(k)}$$

be the scalarized return of a trajectory τ on a held-out dev split (with $w_k(t)$ produced by the in-episode scheduler in Sec. 3.7). We keep an exponential moving average $\hat{J}$ of episode-average returns and trigger the stage change when the improvement over the best running $\hat{J}$ falls below a tolerance ε for P consecutive checkpoints (patience). Intuitively, once exploration plateaus, endpoints are shifted toward efficiency/accuracy for refinement.

Why two stages rather than a fixed two-block schedule. If one sets $w_k^{\text{early}} = w_k^{\text{late}}$ inside each stage, the scheme degenerates to a fixed two-block schedule. Our curriculum controls only the *endpoints*; the actual behavior in each episode is governed by the time-based scheduler below.

3.7 Time-Based Scheduler (within episode)

Progress ratio and interpolation. We schedule step-level weights within each episode using a

progress ratio

$$p(t) = \frac{t-1}{T_{\max}-1} \in [0, 1],$$

where t is the current step and $T_{\max}$ is the episode cap defined in Sec. 3.1. We then linearly interpolate between stage-specific endpoints ($w_k^{\text{early}}, w_k^{\text{late}}$):

$$w_k(t) = (1 - p(t)) w_k^{\text{early}} + p(t) w_k^{\text{late}}.$$

This guarantees $w_k(1) = w_k^{\text{early}}$ and $w_k(T_{\max}) = w_k^{\text{late}}$. If an episode terminates early (ANSWER/REFUSE), the schedule stops at the current t .

By choosing endpoints such that

$$\begin{aligned} w_\beta^{\text{early}} &\geq w_\beta^{\text{late}}, & w_\lambda^{\text{early}} &\geq w_\lambda^{\text{late}}, \\ w_\gamma^{\text{early}} &\leq w_\gamma^{\text{late}}, & w_\eta^{\text{early}} &\leq w_\eta^{\text{late}}, \\ w_\kappa^{\text{early}} &\leq w_\kappa^{\text{late}}, & w_\rho^{\text{early}} &= w_\rho^{\text{late}}, \end{aligned}$$

we ensure exploration incentives (w_β, w_λ) *decay* as evidence accumulates, while efficiency/accuracy ($w_\gamma, w_\eta, w_\kappa$) *increase*; w_ρ stays flat.

Rationale. (i) *Uncertainty reduction*: early steps face high-entropy evidence; rewarding early hits (large w_β) is valuable, but the marginal utility of additional searches diminishes with $p(t)$, so w_β decays. (ii) *Cost–benefit dynamics*: late searches incur growing costs (latency, duplication, context pollution), hence we gradually strengthen overlap/step penalties (w_γ, w_η) and answer accuracy (w_κ). (iii) *Credit assignment*: terminal-only rewards poorly supervise BACKTRACK/REFUSE/STOP; reweighted step signals provide phase-appropriate gradients within the same episode.

Implementation notes. We use linear interpolation for reproducibility; other smooth monotone maps (e.g., sigmoid) are drop-in replacements. Boundary conditions are $w_k(1) = w_k^{\text{early}}$ and $w_k(T_{\max}) = w_k^{\text{late}}$.

4 Experiments

We systematically evaluate EVO-RAG on four prominent multi-hop QA benchmarks: HotpotQA, 2WikiMultiHopQA, MuSiQue, and Bamboogle. Our evaluation specifically targets the following three research questions.

Table 3: Comparison of RAG methods on multi-hop QA datasets. Metrics are EM/F1 (%). For EVO-RAG we report, per backbone, the better of the two reward schedules in Table 4 (Time-dynamic for DeepSeek-8B and Qwen-2.5-7B, Two-stage for LLaMA-3.1-8B). Best value per column in bold.

Method	Backbone	HotpotQA		2Wiki		MuSiQue		Bamboogle	
		EM	F1	EM	F1	EM	F1	EM	F1
RAG-Gym (ReSearch + PRM)	LLaMA-3.1-8B	44.1	56.8	50.2	57.9	48.0	60.0	51.2	63.1
IRCoT	Flan-T5-XXL	45.0	56.2	45.4	56.8	19.9	24.9	44.0	55.0
EVO-RAG	DeepSeek-8B	56.8	70.5	52.6	66.4	51.8	63.7	45.3	58.2
EVO-RAG	LLaMA-3.1-8B	57.4	71.2	53.0	66.9	52.5	64.4	45.7	58.6
EVO-RAG	Qwen-2.5-7B	57.6	71.5	53.2	67.1	52.2	64.0	46.0	59.0

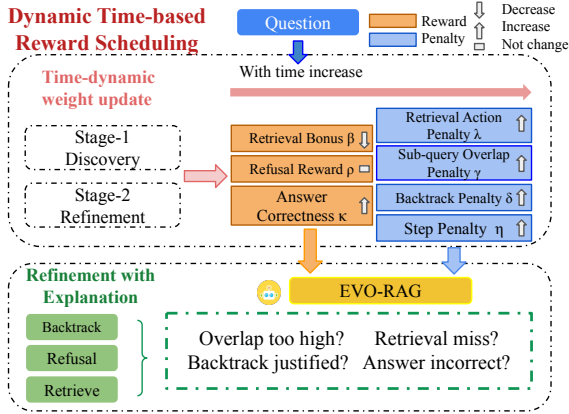


Figure 2: Time-based reward scheduling within an episode. Exploration-oriented weights (β, λ) decrease with progress $p(t)$, efficiency/accuracy weights (γ, η, κ) increase, and the refusal weight (ρ) remains constant.

4.1 Research Questions

Across these benchmarks we focus on three aspects: (i) overall accuracy/efficiency vs. RAG baselines across backbones; (ii) contribution of curriculum/scheduler and the interaction among reward components; (iii) effects of policy objectives (DPO/PPO/GRPO) under identical roll-outs/curricula. Results are summarized in Tables 3, 4, 5, 6, and 7.

4.2 Datasets and Setup

We evaluate EVO-RAG on four multi-hop QA benchmarks (Table 8). All models are trained using 1,000 queries sampled from HotpotQA. Evaluation is conducted on official validation sets. Answer generation is evaluated using Exact Match (EM) and F1 scores. We intentionally keep training confined to HotpotQA to test cross-dataset generalisation.

LLM Backbone and Retriever. We use Llama-3.1-8B-Instruct (Touvron et al., 2023), Qwen2.5-

Table 4: HotpotQA results under different reward schedules. The best schedule per backbone (bold) is the configuration reported in Table 3.

Backbone	Strategy	EM	F1
DeepSeek-8B	No Reward	52.6	66.2
	Two-stage	55.0	68.7
	Time-dynamic	56.8	70.5
LLaMA-3.1-8B	No Reward	52.9	66.6
	Two-stage	57.4	71.2
	Time-dynamic	55.6	69.4
Qwen2.5-7B-Instruct	No Reward	53.1	66.7
	Two-stage	55.9	69.5
	Time-dynamic	57.6	71.5

Table 5: Single-reward ablation. Acc. is the accuracy of the multi-head preference model (Sec. 3.5) at ranking the preferred trajectory prefix on a held-out split when trained with that signal alone; Loss is the corresponding pairwise loss.

Single Reward	Acc. (%)	Loss
Backtrack	70.31	0.913
Refusal	60.58	1.018
Retrieve	55.24	1.089
Step	54.17	1.184
Sub-query Overlap	54.35	1.015

7B-Instruct (Yang et al., 2025), and DeepSeek-R1-Distill-Llama-8B (Guo et al., 2025) as the agent backbones, paired with an RRF-BGE retriever (Chen et al., 2024a) that combines BM25 (Robertson et al., 2009) and BGE dense retrieval. We select these backbones to control for model scale while covering different instruction- and reasoning-oriented model families. LLaMA-3.1-8B-Instruct serves as a strong general-purpose instruction baseline, Qwen2.5-7B-Instruct provides a competitive alternative with strong instruction-following behavior, and DeepSeek-R1-Distill-Llama-8B represents

Table 6: Impact of reward combinations on HotpotQA (Qwen2.5-7B-Instruct). Metrics: EM/F1; Steps is the average retrieval chain length.

Reward Combination	EM	F1	Steps
Baseline (No Reward)	53.1	66.7	8.2
Best-2 (Backtrack + AnsCorr)	56.2	70.0	11.3
Best-3 (+Overlap)	56.9	70.6	10.1
Exploration-Heavy	55.0	69.1	13.4
Efficiency-Heavy	55.4	68.8	9.0
Full (All Rewards)	57.6	71.5	10.4

Table 7: HotpotQA (LLaMA-3.1-8B-Instruct) under different policy objectives, using identical rollouts and curricula. Mean over 3 seeds.

Objective	EM (%)	F1 (%)	Steps
PPO (scalarized)	55.1	69.0	11.7
DPO (multi-pref.)	55.9	69.4	11.1
GRPO (group-norm.)	57.4	71.2	10.2

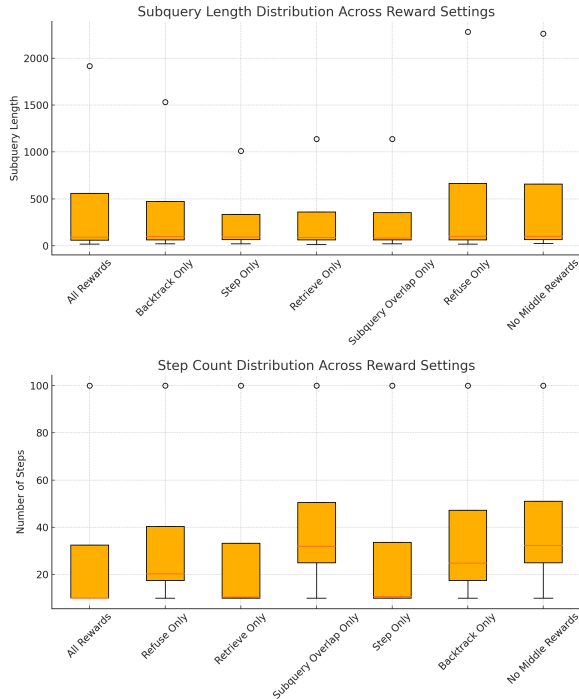


Figure 3: Distribution of sub-query length (left) and step count (right) under different reward settings. Time-dynamic scheduling reduces long-tail behavior compared with exploration-heavy variants.

a reasoning-distilled model. Keeping all three models in the 7B–8B range reduces confounding from model size and makes the effect of reward shaping and policy objectives easier to interpret.

We report refusal frequency and judge outcomes in Table 9 for the full EVO-RAG configuration with time-dynamic scheduling using Qwen2.5-7B-

Table 8: Evaluation data statistics. All models are trained on the same 1,000-query HotpotQA subset and evaluated on the official validation sets.

Dataset	Eval Size	Style
HotpotQA	7,404	Factoid
2WikiMultiHopQA	12,575	Structured
MuSiQue	2,417	Compositional
Bamboogle	125	Adversarial

Table 9: Refusal behavior statistics. Refuse (%) is the share of *all* evaluation questions on which the agent abstains; Valid (%) is the share of *those refusals* judged unanswerable; False (%) is the share of *all* questions on which the agent refuses although the judge deems them answerable.

Dataset	Refuse (%)	Valid (%)	False (%)
HotpotQA	1.9	88	0.2
2Wiki	1.3	85	0.2
MuSiQue	2.4	83	0.4
Bamboogle	0.8	80	0.2

Instruct. Across these mostly answerable benchmarks, refusals are rare, and the false-refusal rate remains low, indicating that REFUSE primarily acts as a safety valve against low-evidence answering rather than a shortcut strategy.

4.3 RQ1: Do we improve over multi-hop RAG baselines?

Main results. Table 3 compares EVO-RAG against RAG-Gym and IRCot across three backbones. On **HotpotQA**, EVO-RAG (Qwen2.5-7B-Instruct) reaches **57.6/71.5**, outperforming RAG-Gym (44.1/56.8) by +13.5 EM / +14.7 F1 and IRCot (45.0/56.2) by +12.6 / +15.3. On **2Wiki**, the best EVO-RAG score (53.2/67.1) exceeds RAG-Gym (50.2/57.9) by +3.0 / +9.2 and IRCot (45.4/56.8) by +7.8 / +10.3. As an exception, **Bamboogle** favors RAG-Gym (51.2/63.1), while EVO-RAG trained only on HotpotQA scores 45–46 / 58–59 (Table 3), indicating that while EVO-RAG generalizes across standard multi-hop QA, it can underperform on adversarially constructed queries without target-domain tuning. Bamboogle contains only 125 items, so a single question shifts EM by 0.8 points; we therefore read this cross-domain gap as indicative rather than conclusive.

Takeaway (RQ1). Across three datasets and multiple backbones, EVO-RAG materially improves EM/F1 over the RAG baselines considered here; the remaining gap on Bamboogle highlights

domain-shift sensitivity for adversarial queries.

4.4 RQ2: What design choices matter—components and scheduling?

(a) **Single-reward ablation (component strength).** Table 5 isolates each signal and reports internal decision quality (evaluation accuracy/loss of the preference model; see Sec. 3.4 and Sec. 3.5). **Backtrack** alone yields the highest internal accuracy (70.31%), indicating that controlled reversibility is a strong driver for robust exploration. **Refusal** ranks second (60.58%), supporting our design to explicitly reward safe abstention when evidence is insufficient. Pure **Retrieve/Step/Overlap** signals are weaker in isolation, suggesting they are most effective in combination rather than alone.

Takeaway. Signals that regulate *control flow* (when to backtrack or refuse) carry disproportionate value; more local efficiency signals need to be paired with them.

(b) **Reward combinations and scheduling (Two-stage vs. Time-dynamic).** *Combinations.* We use Qwen2.5-7B-Instruct for the reward-combination study because it gives the strongest overall HotpotQA performance under the full EVO-RAG setting in Table 4, making it the most informative setting for isolating which reward subsets matter. On HotpotQA (Qwen2.5-7B-Instruct), Table 6 shows that moving from Baseline (No Reward) (53.1/66.7, 8.2 steps) to **Best-2** (Backtrack+AnswerCorrectness) already gives +3.1 EM; adding **Overlap (Best-3)** both increases EM/F1 (56.9/70.6) and shortens chains (10.1 vs. 11.3). The **Full** configuration (all rewards, time-dynamic) yields the best accuracy **57.6/71.5** at 10.4 steps—longer than Baseline but substantially more accurate, indicating a better accuracy–efficiency trade-off. *Exploration-Heavy* extends chains (13.4 steps) with lower EM (55.0); *Efficiency-Heavy* shortens chains (9.0) but loses EM (55.4).

Schedules. Table 4 compares No Reward, Two-stage, and Time-dynamic: **DeepSeek-R1-Distill-Llama-8B:** Time-dynamic > Two-stage > No-Reward (56.8/70.5 vs. 55.0/68.7 vs. 52.6/66.2), i.e., +1.8/+1.8 over Two-stage and +4.2/+4.3 over No-Reward. **Qwen2.5-7B-Instruct:** Time-dynamic likewise wins (57.6/71.5 vs. 55.9/69.5 vs. 53.1/66.7), i.e., +1.7/+2.0 and +4.5/+4.8. **LLaMA-3.1-8B:** Two-stage slightly outperforms Time-dynamic (57.4/71.2 vs. 55.6/69.4), while both clearly beat No-Reward (52.9/66.6).

Interpretation. The in-episode scheduler consistently helps (DeepSeek/Qwen), while LLaMA-3.1-8B-Instruct appears to benefit more from fixed stage endpoints. This suggests a backbone–schedule interaction: when the model’s search policy is already stable, a smoother decay (Time-dynamic) prevents over-search; otherwise, a stiffer stage separation (Two-stage) may be easier to learn. Figure 3 corroborates the efficiency story: dynamic scheduling suppresses long tails in step counts and sub-query lengths.

Takeaway (RQ2). Component-wise, **Backtrack + AnswerCorrectness (+Overlap)** form a strong core; curriculum-wise, the Time-dynamic scheduler is generally superior, with a small exception on LLaMA-3.1-8B-Instruct, where Two-stage wins by a narrow margin.

4.5 RQ3: How do policy objectives compare (DPO vs. PPO vs. GRPO)?

For the objective comparison, we use LLaMA-3.1-8B-Instruct as a stable reference backbone because it exhibits competitive performance under both schedules (Table 4) and provides a clean setting for comparing DPO, PPO, and GRPO under identical rollouts.

Observed trends. Table 7 reports the three objectives under identical rollouts and curricula, averaged over three seeds. GRPO attains both the highest accuracy and the shortest chains, DPO is intermediate, and PPO trails on both axes. We read this as a variance-structure effect rather than a general ranking: group normalization helps most when sibling actions at a given state receive similar scalarized rewards, DPO sidesteps scalarization altogether and is therefore robust when reward weights are hard to tune, and PPO can match either when the weights are well calibrated. Because we measure a single dataset/backbone pair, we do not claim a universal ordering.

Takeaway (RQ3). On HotpotQA with LLaMA-3.1-8B-Instruct we observe **GRPO > DPO > PPO** in both accuracy and efficiency (Table 7); the preferred objective may vary with dataset/backbone and with the variance structure of sibling actions.

5 Conclusion

We presented EVO-RAG, a phase-aware framework for multi-hop agentic RAG that combines (i) a lightweight Discovery→Refinement curriculum, (ii) seven interpretable step-level signals, and

(iii) an in-episode time scheduler that shifts emphasis from exploration to efficiency and correctness as evidence accumulates. Beyond scalar rewards, we introduced a multi-head preference model that preserves interpretability while enabling preference alignment (DPO) and scalarized policy gradients (PPO/GRPO) under identical rollouts for controlled comparisons. Across HotpotQA, 2WikiMultiHopQA, and MuSiQue, EVO-RAG consistently improves EM/F1 over the representative baselines and internal variants considered in this study, while reducing redundant behavior. Ablations further highlight the importance of control-flow signals and time-dynamic weighting. Future work includes broader baseline coverage, deeper and more diverse benchmarks, and learning adaptive schedulers and refusal validation with stronger human-grounded evaluation.

Limitations

Our empirical comparison focuses on RAG-Gym and IRCot as representative multi-hop RAG baselines. Recent search-augmented or RL-trained RAG agents (e.g., R1-Searcher, O²-Searcher, FrugalRAG, SimpleDeepSearcher, Chain-of-Retrieval Augmented Generation) are not included in our experimental tables. This limits how strongly we can claim competitiveness against the latest systems; our primary goal is instead a controlled study of phase-aware step signals and objective choices under identical rollouts.

We evaluate on four Wikipedia-style multi-hop QA datasets (HotpotQA, 2WikiMultiHopQA, MuSiQue, Bamboogle), which are mostly 2–4-hop in nature. Performance on deeper reasoning tasks, longer-horizon search, or non-Wikipedia domains remains uncertain. The drop on adversarial Bamboogle further suggests sensitivity to domain shift and dataset construction artifacts; with only 125 evaluation items, differences on this benchmark should be interpreted cautiously.

Reward weights and interpolation endpoints are manually specified (Table 2). While we provide a pilot study on adaptive tuning, our main method does not learn the scheduler itself. In addition, the Discovery→Refinement curriculum assumes a roughly monotone transition from exploration to consolidation; tasks that require intermittent re-exploration may not perfectly match this linear trajectory.

The refusal reward relies on an external LLM

judge to label answerable vs. unanswerable. This introduces potential bias, judge-model dependency, and misalignment with human judgments. We do not report a dedicated human study for refusal validity, which is important for understanding the reliability of abstention behaviors.

Acknowledgments

FILL IN: funding sources, grant numbers, and the compute provider for the A100 runs reported in Table 12.

References

- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, and 12 others. 2020. [Language models are few-shot learners](#). In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc.
- Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024a. Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2402.03216*.
- Jiawei Chen, Hongyu Lin, Xianpei Han, and Le Sun. 2024b. Benchmarking large language models in retrieval-augmented generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 17754–17762.
- Yiqun Chen, Lingyong Yan, Weiwei Sun, Xinyu Ma, Yi Zhang, Shuaiqiang Wang, Dawei Yin, Yiming Yang, and Jiaxin Mao. 2025. Improving retrieval-augmented generation through multi-agent reinforcement learning. *arXiv preprint arXiv:2501.15228*.
- Jingsheng Gao, Linxu Li, Weiyuan Li, Yuzhuo Fu, and Bin Dai. 2024. Smartrag: Jointly learn rag-related tasks from the environment feedback. *arXiv preprint arXiv:2410.18141*.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, and 1 others. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.
- Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. [Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps](#). In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 6609–6625, Barcelona, Spain (Online). International Committee on Computational Linguistics.

- Jerry Huang, Siddarth Madala, Risham Sidhu, Cheng Niu, Julia Hockenmaier, and Tong Zhang. 2025. Rag-rl: Advancing retrieval-augmented generation via rl and curriculum learning. *arXiv preprint arXiv:2503.12759*.
- Magdalena Kaiser and Gerhard Weikum. 2025. Preference-based learning with retrieval augmented generation for conversational question answering. *arXiv preprint arXiv:2503.22303*.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, and 1 others. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474.
- Tianci Liu, Haoxiang Jiang, Tianze Wang, Ran Xu, Yue Yu, Linjun Zhang, Tuo Zhao, and Haoyu Wang. 2025. Roserag: Robust retrieval-augmented generation with small-scale llms via margin-aware preference optimization. *arXiv preprint arXiv:2502.10993*.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, and 1 others. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744.
- Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, and Mike Lewis. 2022. Measuring and narrowing the compositionality gap in language models. *arXiv preprint arXiv:2210.03350*.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67.
- Shyam Sundhar Ramesh, Yifan Hu, Iason Chaimalas, Viraj Mehta, Pier Giuseppe Sessa, Haitham Bou Ammar, and Ilija Bogunovic. 2024. Group robust preference optimization in reward-free rlhf. *Advances in Neural Information Processing Systems*, 37:37100–37137.
- Stephen Robertson, Hugo Zaragoza, and 1 others. 2009. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends® in Information Retrieval*, 3(4):333–389.
- Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. 2025. R1-searcher: Incentivizing the search capability in llms via reinforcement learning. *arXiv preprint arXiv:2503.05592*.
- Zhongxiang Sun, Qipeng Wang, Weijie Yu, Xiaoxue Zang, Kai Zheng, Jun Xu, Xiao Zhang, Song Yang, and Han Li. 2025. Rearter: Retrieval-augmented reasoning with trustworthy process rewarding. *arXiv preprint arXiv:2501.07861*.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, and 1 others. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022a. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. *arXiv preprint arXiv:2212.10509*.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022b. Musique: Multi-hop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554.
- Zilong Wang, Zifeng Wang, Long Le, Huaixiu Steven Zheng, Swaroop Mishra, Vincent Perot, Yuwei Zhang, Anush Mattapalli, Ankur Taly, Jingbo Shang, and 1 others. 2024. Speculative rag: Enhancing retrieval augmented generation through drafting. *arXiv preprint arXiv:2407.08223*.
- Zhepei Wei, Wei-Lin Chen, and Yu Meng. 2024. Instrutrag: Instructing retrieval-augmented generation via self-synthesized rationales. *arXiv preprint arXiv:2406.13629*.
- Guangzhi Xiong, Qiao Jin, Xiao Wang, Yin Fang, Haolin Liu, Yifan Yang, Fangyuan Chen, Zhixing Song, Dengyu Wang, Minjia Zhang, and 1 others. 2025. Rag-gym: Optimizing reasoning and search agents with process supervision. *arXiv preprint arXiv:2502.13957*.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. *arXiv preprint arXiv:1809.09600*.
- Nan Zhang, Prafulla Kumar Choubey, Alexander Fabri, Gabriel Bernadett-Shapiro, Rui Zhang, Prasenjit Mitra, Caiming Xiong, and Chien-Sheng Wu. 2024. Sirerag: Indexing similar and related information for multihop reasoning. *arXiv preprint arXiv:2412.06206*.
- Rongzhi Zhu, Xiangyu Liu, Zequn Sun, Yiwei Wang, and Wei Hu. 2025. Mitigating lost-in-retrieval problems in retrieval augmented multi-hop question answering. *arXiv preprint arXiv:2502.14245*.

A Adaptive Reward Weight Tuning (Pilot Study)

As a feasibility check for learning the schedule rather than specifying it by hand, we compare our manual endpoints against Bayesian optimization (BO) and a bandit-based (UCB1) controller over the same weight space (Table 10). Adaptive methods yield comparable or slightly better accuracy than the manual schedule. BO converged to weights close to our hand-tuned configuration with marginal EM/F1 gains; UCB1 adapted weights online without manual intervention. These results support the feasibility of adaptive tuning for robustness and cross-domain transfer (e.g., MuSiQue, Bamboogle).

Table 10: Pilot study of adaptive reward weight tuning on HotpotQA (LLaMA-3.1-8B-Instruct).

Method	EM (%)	F1 (%)	Steps
Manual (main paper)	57.4	71.2	10.2
Bayesian opt. (BO)	57.8	71.5	10.1
Bandit (UCB1)	57.6	71.3	10.4

B Parameters used in VERL

Table 11 lists the training hyperparameters for each policy objective.

C Compute and Cost

Table 12 reports the training overhead of EVO-RAG. Reward computation is lightweight at inference time; the extra cost is incurred during training due to rollout generation, preference modeling, and optional LLM-based refusal validation.

D Case Studies

Case study interpretation. Across Q1–Q2, the baseline repeatedly issues paraphrased queries, which inflates steps and often misses the key entity. Two-stage reduces duplication but can still over-search late. Time-dynamic scheduling increases overlap/step pressure as $p(t)$ grows, leading to earlier consolidation and fewer redundant hops. For Q3 (unanswerable), the refusal signal prevents the agent from hallucinating a plausible location and terminates with REFUSE.

Case Q1 (answerable, entity linking). **Question trait:** 2-hop bridging question; the second hop depends on resolving an ambiguous entity mention (alias/name collision). **Baseline failure:** the

baseline keeps issuing paraphrased sub-queries that preserve the same ambiguous surface form, leading to near-duplicate retrievals and repeated evidence; after a miss, it continues searching late instead of revising the entity. **EVO-RAG turning point:** the overlap penalty increases as $p(t)$ grows, forcing a lexical/semantic pivot in the query (entity disambiguation cue added); after the first miss, the agent executes a BACKTRACK and rewrites the query conditioned on the most informative snippet, which yields the missing bridge passage. **Outcome:** fewer redundant hops (steps ↓), earlier hit on the supporting document, and the final answer is correct (EM/F1 ↑).

Case Q2 (answerable, multi-evidence consolidation). **Question trait:** 3–4 hop compositional QA requiring synthesizing two complementary facts from different pages (e.g., entity attribute + event relation); intermediate evidence is partially relevant but not sufficient alone. **Baseline failure:** the baseline over-explores late: after already retrieving the key supporting passages, it continues to issue additional searches with high overlap, which adds distractor snippets and degrades the final generation (context pollution / delayed stopping). **EVO-RAG turning point:** as $p(t)$ increases, the step penalty and retrieval-action penalty dominate, making late SEARCH costly; the agent stops searching once redundancy rises and instead commits to ANSWER using the already retrieved evidence (controlled stopping). **Outcome:** average steps decrease while maintaining or improving correctness; the answer is supported by the consolidated evidence (EM/F1 ↑; steps ↓).

Case Q3 (unanswerable, safe refusal). **Question trait:** unanswerable under the available corpus / retrieved context (missing required entity or the question presupposes a false premise); retrieval returns plausible but non-supporting snippets. **Baseline failure:** the baseline attempts to answer anyway by extrapolating from loosely related passages, producing a fluent but unsupported response (hallucination). **EVO-RAG turning point:** the policy selects REFUSE after repeated retrieval misses / low-support evidence; the refusal validity reward (LLM judge) verifies insufficient evidence and reinforces abstention rather than speculative answering. **Outcome:** the agent correctly abstains (REFUSE is judged valid), avoiding hallucination; refusal correctness ↑ and unsafe answers ↓.

Table 11: VERL training hyperparameters by objective (shared unless noted).

Hyperparameter	PPO	GRPO
Critic model path	Qwen2.5-0.5B-Instruct	—
LR (actor / critic)	1e−6 / 1e−5	1e−6 / —
Train batch size (episodes)	128	16
PPO mini-batch size	64	8
Micro-batch / GPU	4	2
Max prompt / resp. length	2048 / 256	1330 / 256
KL coef (λ_{KL})	0.001	0.0
Adv estimator	PPO (GAE)	GRPO
TP size (vLLM)	1	2
vLLM gpu_mem util	0.4	0.4
Rollout n (per prompt)	—	1
Critic warmup	—	0
Epochs / save / test freq	15 / 10 / 10	15 / 10 / 10
GPUs (per node)	1	2
Seeds	3	

Table 12: Compute cost breakdown (3 seeds; $\approx$ marks approximate values). Policy-optimization hours are reported as PPO / DPO / GRPO.

Component	Quantity	Value
Rollout generation	GPU	A100-80G
	Episodes	1000 $\times$ 3 seeds
	GPU-hours	13.0
Preference model	Updates	$\approx$ 6k
	GPU-hours	$\approx$ 1.6
Policy optimization	GPU-hours	$\approx$ 3.8 / 1.2 / 2.4
Refusal judge	Calls	$\approx$ 60
	Latency	$\approx$ 1.2 s / call
	Tokens	$\approx$ 350 / call

E Refusal Judge Prompt

Input: question x , retrieved snippets $\{d_1, \dots, d_k\}$, and the agent’s proposed refusal.

Task: decide whether the question is answerable using only the provided evidence. Output one token: answerable or unanswerable.

```

You are a strict evaluator for
evidence-based answerability.

[Question] {QUESTION}
[Retrieved Evidence] {SNIPPETS}

Task: Decide whether the question is
answerable using ONLY the retrieved
evidence above. Do NOT use outside
knowledge. Do NOT guess. The evidence
may be partially relevant or misleading.

Decision Rule: Return answerable iff
the evidence contains enough information
to answer the question correctly and
specifically. Return unanswerable if key
facts are missing, contradictory, or the
evidence only supports a vague guess.

Output: Output exactly one token:
answerable or unanswerable.

```